

# Self-Attention & Transformers

SMM694: Natural Language Processing

Session 5 | Valerio Modugno

---

# The Necessity of Recurrence

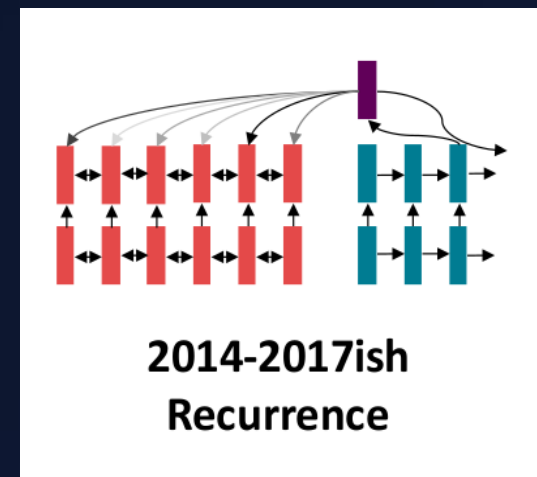
## Do we truly need RNNs?

For years, Recurrent Neural Networks (RNNs) were the undisputed standard for processing sequential data in NLP.

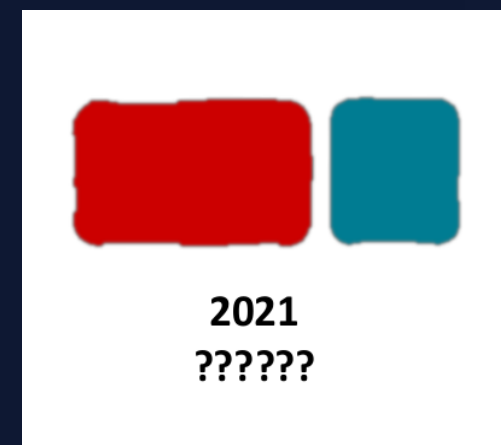
However, we must question their absolute necessity.

Abstractly, attention mechanisms serve the exact same foundational purpose as RNNs: effectively passing information from a sequence to a neural network input.

If attention can route this information more directly, recurrence itself might be an architectural bottleneck rather than a requirement.



lots of trial and error



# A Paradigm Shift in NLP

2017 - 2020

The introduction of the Transformer model completely disrupted the standard, proving attention alone is sufficient.

2014 - 2017

Heavy reliance on Recurrence. The field advanced largely through trial and error with complex LSTM and GRU architectures.

2021+

Total paradigm shift. New model architectures dominated by attention mechanisms scale to billions of parameters, leaving traditional RNNs behind.

# Cross-Attention vs. Self-Attention

## Cross-Attention

Previously discussed in the context of sequence-to-sequence models (like translation). It involves a decoder paying attention to a completely separate encoder sequence.

While powerful, it requires an established input sequence to act as the permanent reference point for the decoding process.

## Self-Attention

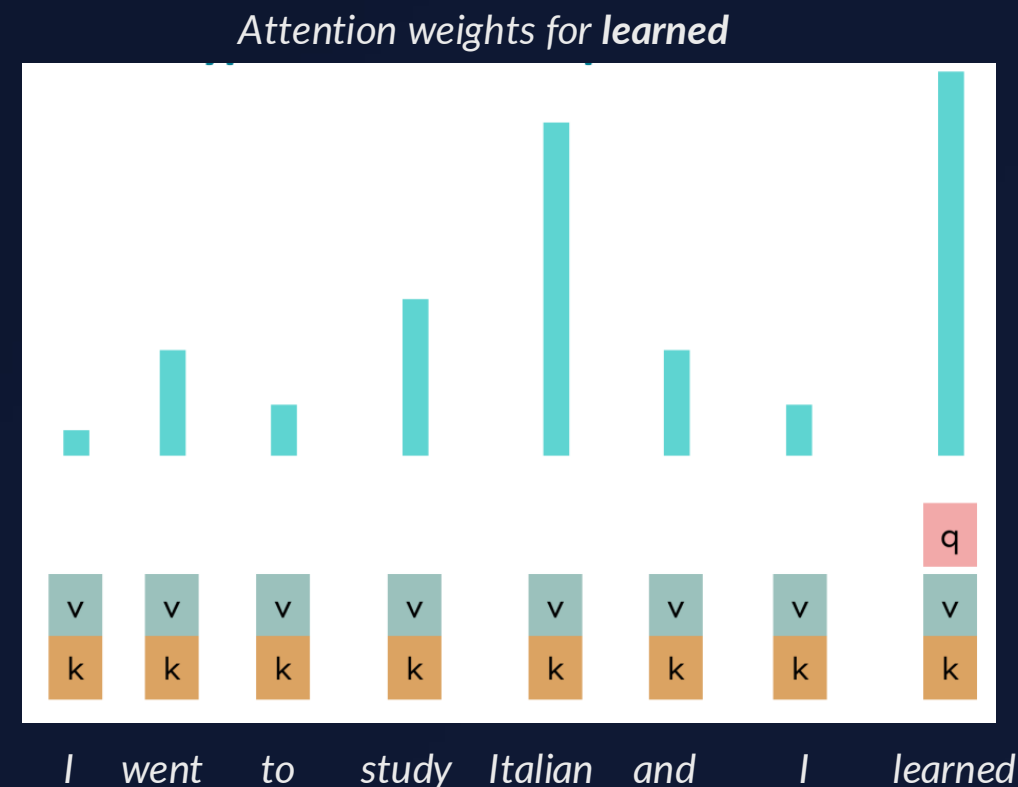
The essential, foundational building block for autonomous sequence generation. Instead of looking at an external source, the model looks at itself. It requires the model to dynamically pay attention to its own past outputs  $y_{<t}$ . In order to generate the precise current output  $y_t$ .

# Self-Attention in Action

## Hypothetical Example

Consider the sentence: *"I went to study Italian and I learned"*

- ▶ To understand how to process the word "learned", the network utilizes distinct roles for each word.
- ▶ **Queries (Q):** What information am I looking for?
- ▶ **Keys (K):** What information do I possess?
- ▶ **Values (V):** What is the actual content I will transmit if matched?
- ▶ By evaluating these blocks across the sequence, the model dynamically routes context.



# Highlighting Attention Weights



## The Target Token

Continuing our example, the target word is "learned". The model generates a Query vector specifically for this token to seek out relevant context from the past.



## Key Interactions

The Query for "learned" interacts with the Keys of all preceding words (e.g., "Italian", "study", "in"). The dot product calculates the specific affinity or "weight" of that connection.



## Weight Distribution

Visually, this creates a bar chart of probabilities. Words like "Italian" and "study" will receive significantly higher attention weights than structural words like "and" or "to".

# Mathematical Formulation

Let  $\mathbf{w}_{1:n}$  be a sequence of words in vocabulary  $V$ , like *Zuko made his uncle tea*.

For each  $\mathbf{w}_i$ , let  $\mathbf{x}_i = E\mathbf{w}_i$ , where  $E \in \mathbb{R}^{d \times |V|}$  is an embedding matrix.

1. Transform each word embedding with weight matrices  $Q, K, V$ , each in  $\mathbb{R}^{d \times d}$

$$\mathbf{q}_i = Q\mathbf{x}_i \text{ (queries)} \quad \mathbf{k}_i = K\mathbf{x}_i \text{ (keys)} \quad \mathbf{v}_i = V\mathbf{x}_i \text{ (values)}$$

2. Compute pairwise similarities between keys and queries; normalize with softmax

$$\mathbf{e}_{ij} = \mathbf{q}_i^\top \mathbf{k}_j \quad \alpha_{ij} = \frac{\exp(\mathbf{e}_{ij})}{\sum_{j'} \exp(\mathbf{e}_{ij'})}$$

3. Compute output for each word as weighted sum of values

$$\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_i$$

# Attention Mechanism in a Nutshell

a fluffy blue creature roamed the verdant forest



# Major Barrier 1: Sequence Order

||

Pure self-attention acts entirely as a  
bag-of-words model. It  
fundamentally lacks any inherent  
notion of sequence order or syntax.

||

— The Positional Limitation

---

# Solving the Order Problem

## Position Vectors

Because attention operations are symmetric and order-agnostic, we must artificially inject sequence order directly into the data itself.

We introduce position vectors  $p_i \in \mathbb{R}^d$ . These vectors contain mathematical signatures representing absolute or relative positions in the text.

By adding these vectors to the original word embeddings at the very first layer, we yield position-aware embeddings:

$$\tilde{x}_i = x_i + p_i$$

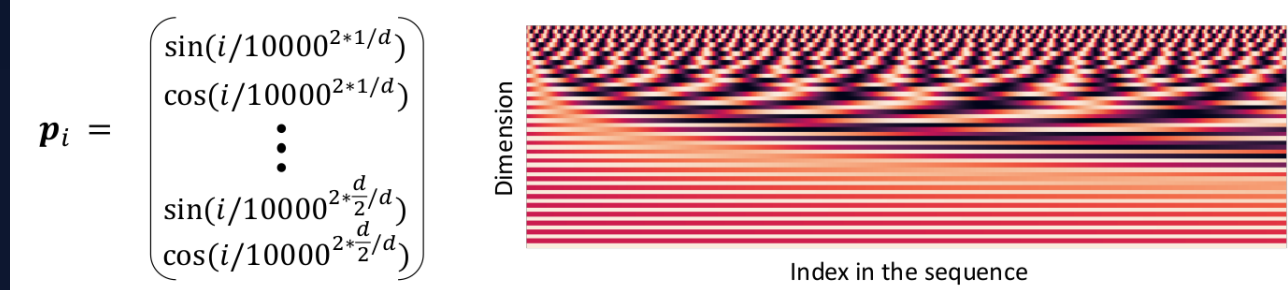
In deep self-attention networks, we do this at the first layer! You could concatenate them as well, but people mostly just add...

# Sinusoidal Representations

## The Mechanism

The original Transformer paper proposed using concatenated sine and cosine functions of varying frequencies to encode positional data.

This creates a continuous, deterministic geometric progression that provides a unique mathematical signature for every discrete position index in the sequence, offering strong periodicity.



## The Limitations

While conceptually elegant, these representations are completely fixed and not learnable by the neural network during training.

# Learned Absolute Position

## Flexible Parameter Matrices

As an alternative to fixed sinusoids, modern implementations often utilize learned absolute position representations.

In this setup, every positional index  $p_i$  is a directly learnable parameter housed within a massive embedding matrix. The model learns the optimal representation for "position 5" versus "position 10".

While this is highly flexible and widely used, it possesses a hard constraint: the model absolutely cannot extrapolate to indices outside the strictly defined maximum sequence length seen in training.

$$p = \begin{bmatrix} p_{1,1} & p_{1,2} & \dots & p_{1,n} \\ p_{2,1} & p_{2,2} & \dots & p_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ p_{d,1} & p_{d,2} & \dots & p_{d,n} \end{bmatrix}$$

---

# Rotary Position Embedding (ROPE)

A high-level paradigm shift: Embeddings should rely on the relative distance  $(i - j)$  between tokens, overcoming the strict failures of absolute embeddings.

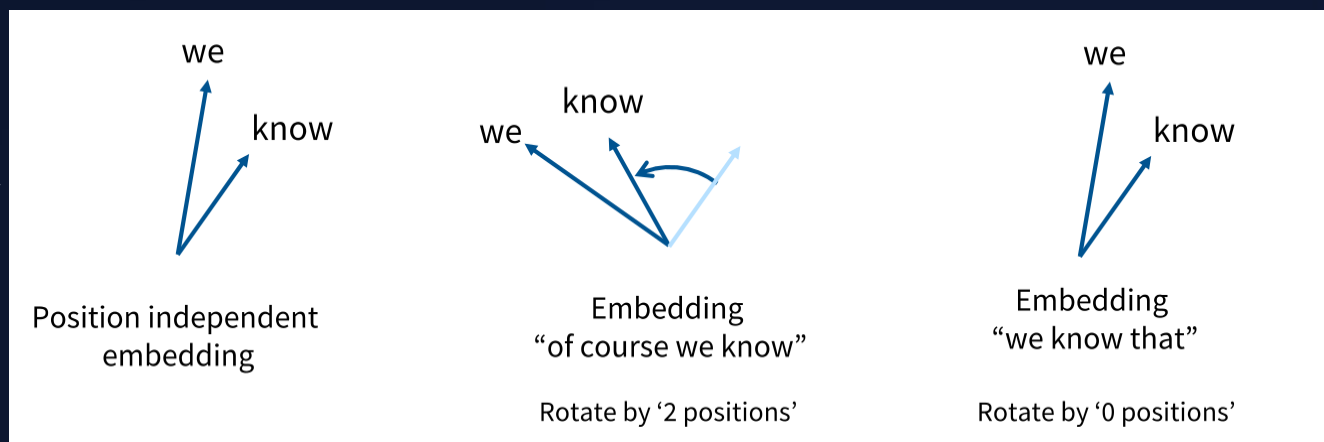
---

# The Conceptual Solution of ROPE

## Mathematical Invariance

The goal of ROPE is to make the attention mechanism fundamentally invariant to absolute position shifts, caring only about distance.

It achieves this elegant solution by leveraging a foundational fact of linear algebra: inner products between vectors are naturally invariant to arbitrary, uniform geometric rotations. By defining position as a rotation angle rather than a static vector addition, the model implicitly understands relative token distances.



$$\langle f(x, i), f(y, j) \rangle = g(x, y, i - j)$$

# Scaling ROPE to High Dimensions



## Pairing Coordinates

To scale beyond simple 2D vectors, high-dimensional word embeddings are split into discrete pairs of coordinates across their entire length.



## Applying Rotation

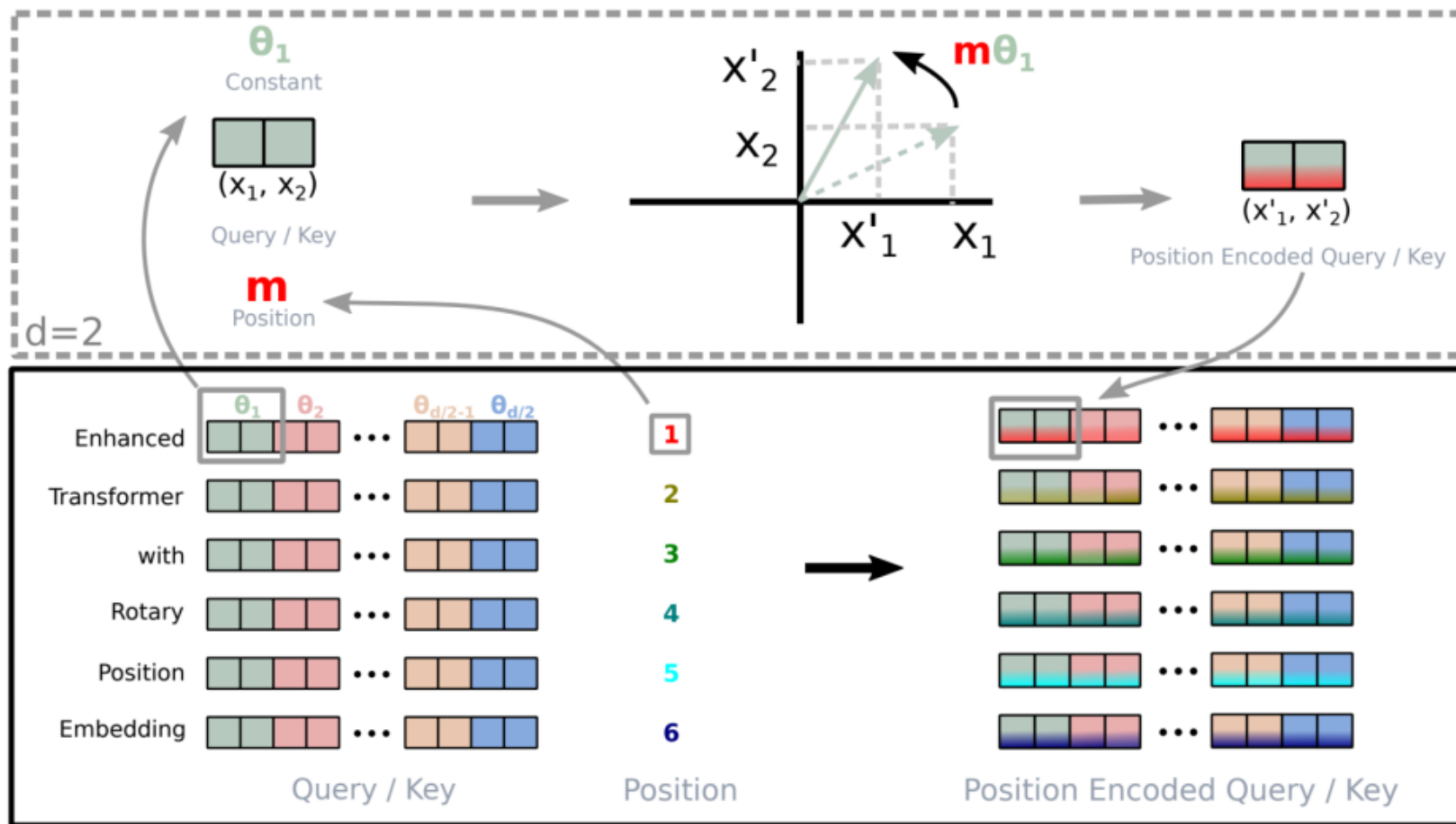
A specific mathematical rotation matrix, determined by the token's absolute position, is independently applied to each of these coordinate pairs.



## Relative Encoding

When computing attention, the dot products of these rotated vectors magically cancel out the absolute angles, leaving an interaction term dependent strictly on their relative distance.

# Scaling ROPE to High Dimensions



[Su et al 2021]

Just pair up the coordinates and rotate them in 2d (motivation: complex numbers)

# Barrier 2: Nonlinearities

## 02

Crucial Bottleneck

### The Linearity Problem

Having addressed position, we must confront a second major problem: pure self-attention is entirely linear with respect to the value vectors.

The output of the attention mechanism is nothing more than a series of weighted averages. Deep learning relies fundamentally on complex representations.

Without incorporating elementwise nonlinearities (like ReLU or GELU) inside a feed-forward network, stacking multiple attention layers yields no additional representational power.

# Solving the Nonlinearity Barrier

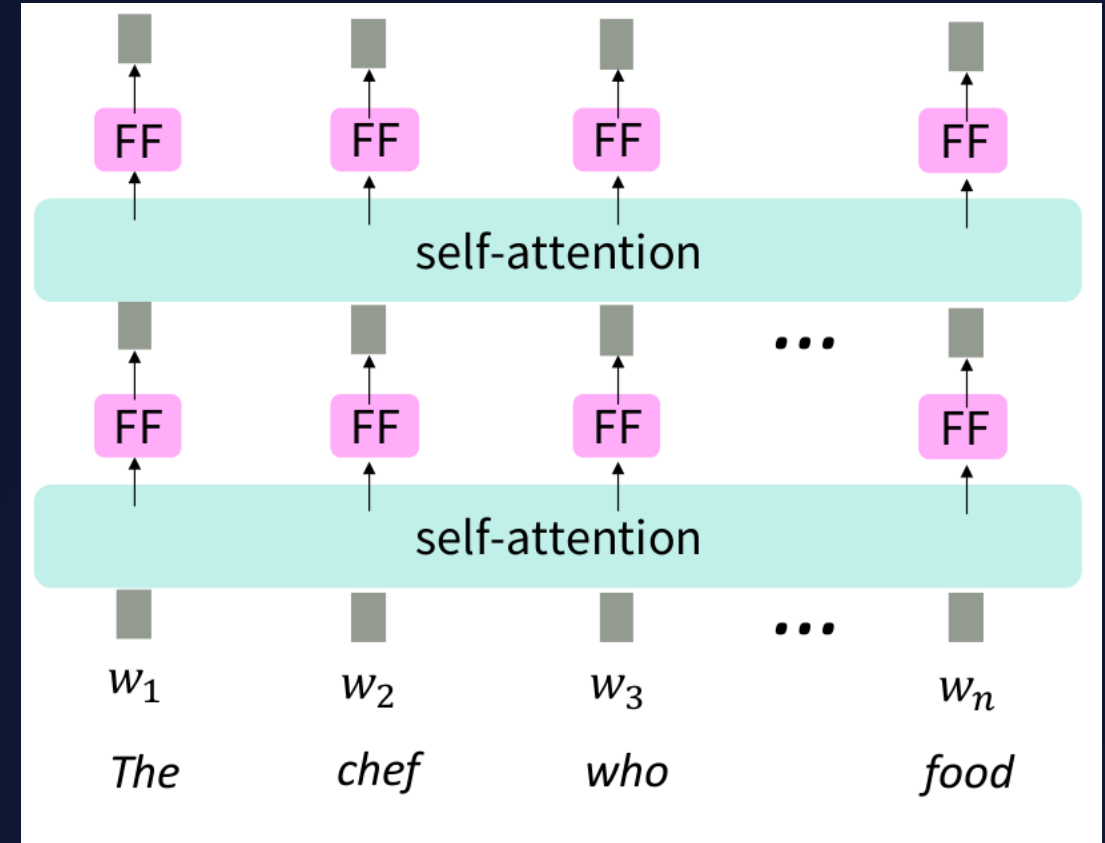
## Adding the Feed-Forward Network

Pure self-attention acts merely as a weighted average.

Without nonlinear activations, deep stacking provides no additional representational power.

To break this linearity, we introduce a Feed-Forward (FF) network to independently post-process each output vector using a ReLU activation function.

$$\begin{aligned} m_i &= MLP(\text{output}_i) \\ &= W_2 * \text{ReLU}(W_1 \text{output}_i + b_1) + b_2 \end{aligned}$$



## Barrier 3: Predicting the Future

||

When predicting a sequence in decoders, the model must be strictly prevented from looking at the future tokens to maintain autoregressive integrity.

||

— The Causality Constraint

---

# The Masking Solution

## Preventing "Future-Peeking"

In sequence generation, an attention mechanism running across all tokens would allow the model to trivially "cheat" by looking at the actual target word it is supposed to predict.

We need a mechanism that enforces a strict unidirectional flow of information, allowing a word to only attend to itself and its predecessors.

|         | [START] | The       | chef      | who       |
|---------|---------|-----------|-----------|-----------|
| [START] |         | $-\infty$ | $-\infty$ | $-\infty$ |
| The     |         |           | $-\infty$ | $-\infty$ |
| chef    |         |           |           | $-\infty$ |
| who     |         |           |           |           |

## Implementation via Negative Infinity

To solve this while retaining the speed of parallel matrix operations, we apply an attention mask. We artificially set the pre-softmax attention scores of all future words to  $-\infty$ .

# Overcoming Self-Attention Barriers



## Sequence Order

Barrier: No inherent notion of order.

Solution: Add structural position representations (embeddings) directly into the input tokens at the first layer.



## Linearity Problem

Barrier: Operations are purely linear weighted averages.

Solution: Post-process the attention outputs with a non-linear Feed-Forward network utilizing ReLU.



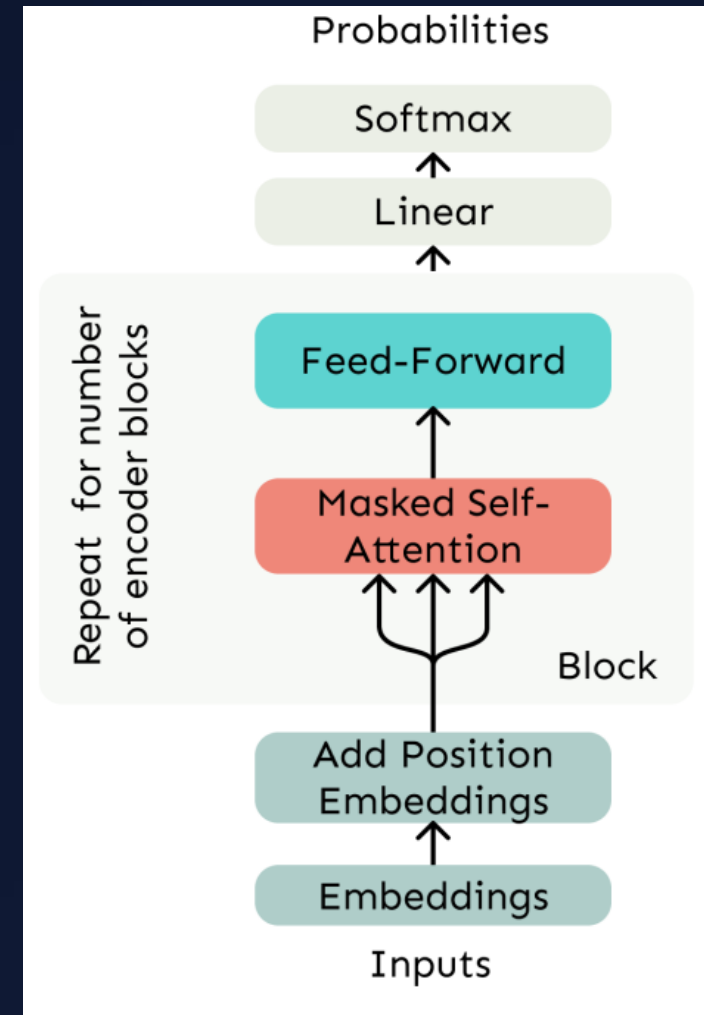
## Future Peeking

Barrier: Decoders cannot view future words during generation.

Solution: Apply causal masking to force future attention interaction weights strictly to zero.

# Self-Attention Block Checklist

- ✓ **Self-Attention:** The core mechanism allowing sequence tokens to interact dynamically and share contextual meaning.
- ✓ **Position Representations:** The injected signals that provide mathematical context regarding spatial order and relative distance.
- ✓ **Nonlinearities:** The Feed-Forward networks that allow the deep learning architecture to map complex, non-linear representations.
- ✓ **Masking:** The absolute causality requirement necessary for autoregressive decoders to function without cheating during training.



---

# The Transformer Model

Assembling the components into a groundbreaking sequence  
architecture.

---

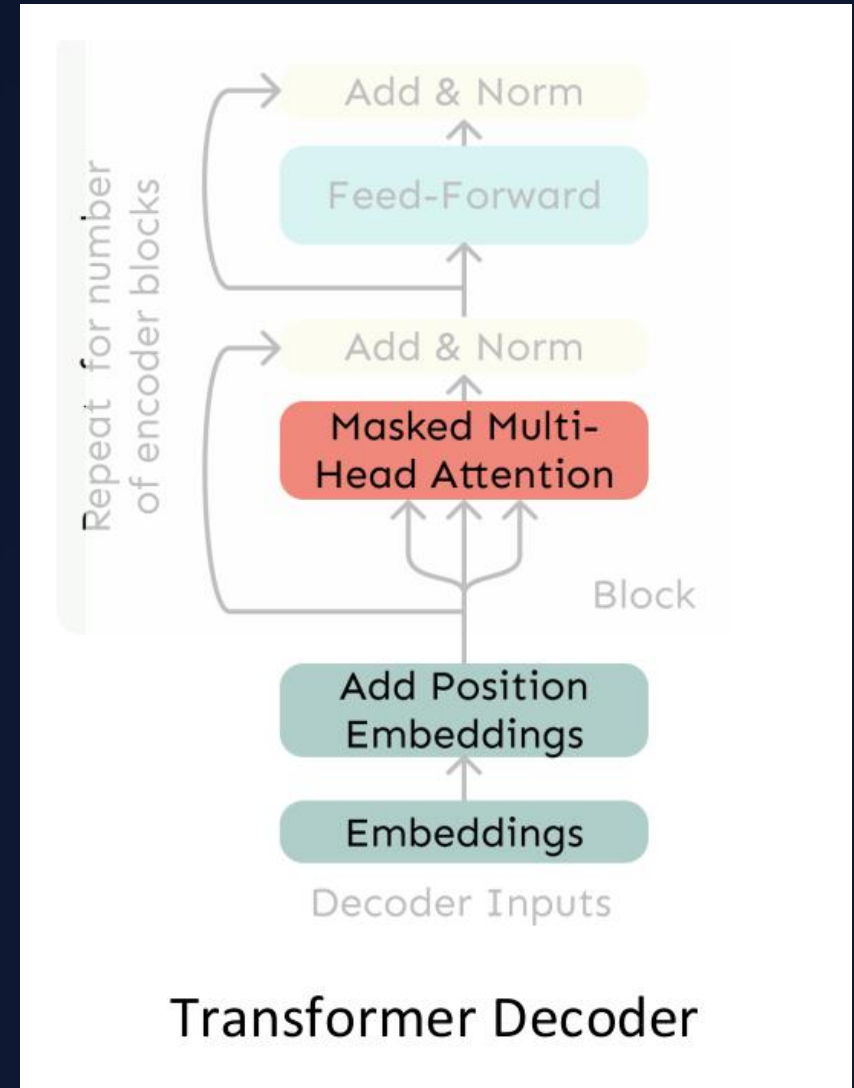
# Transformer Decoder Architecture

## Assembling the Stack

The full Transformer Decoder stack heavily relies on the minimal self-attention architecture we have built so far.

It ingests embeddings and positional embeddings, passing them into the core attention block.

However, we are now upgrading the standard operation to Masked Multi-Head Attention, allowing the network to simultaneously route different types of distinct information streams.

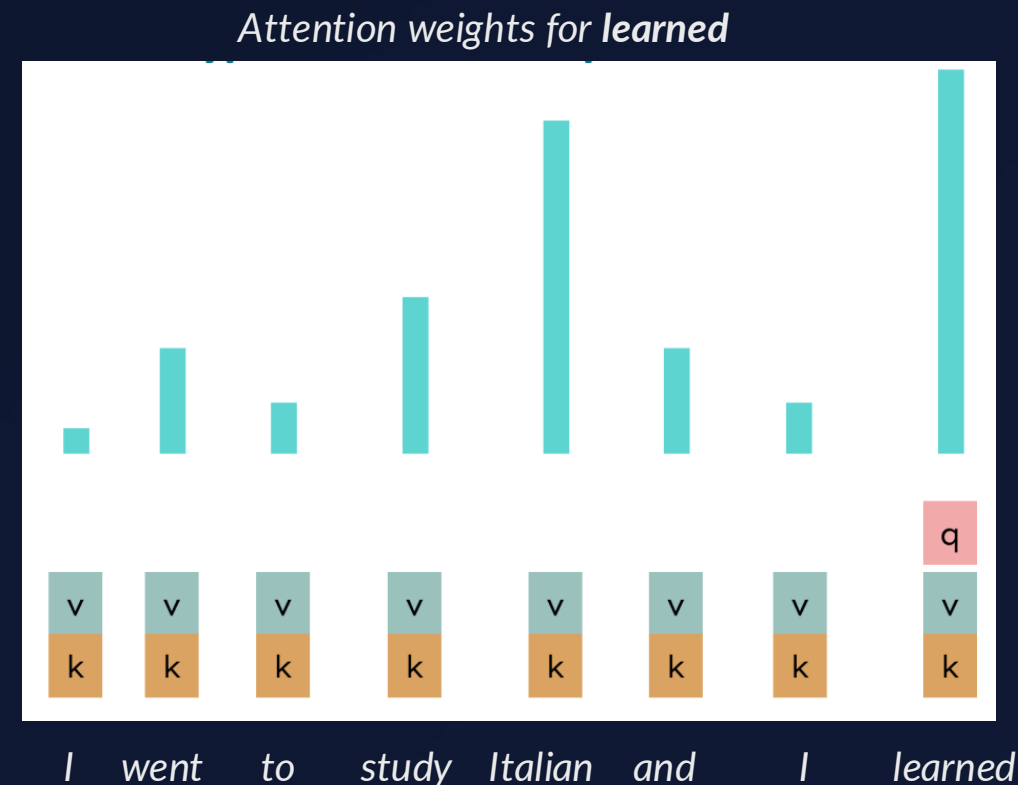


# Primer for Multi-Head Attention

## Expanding the Focus

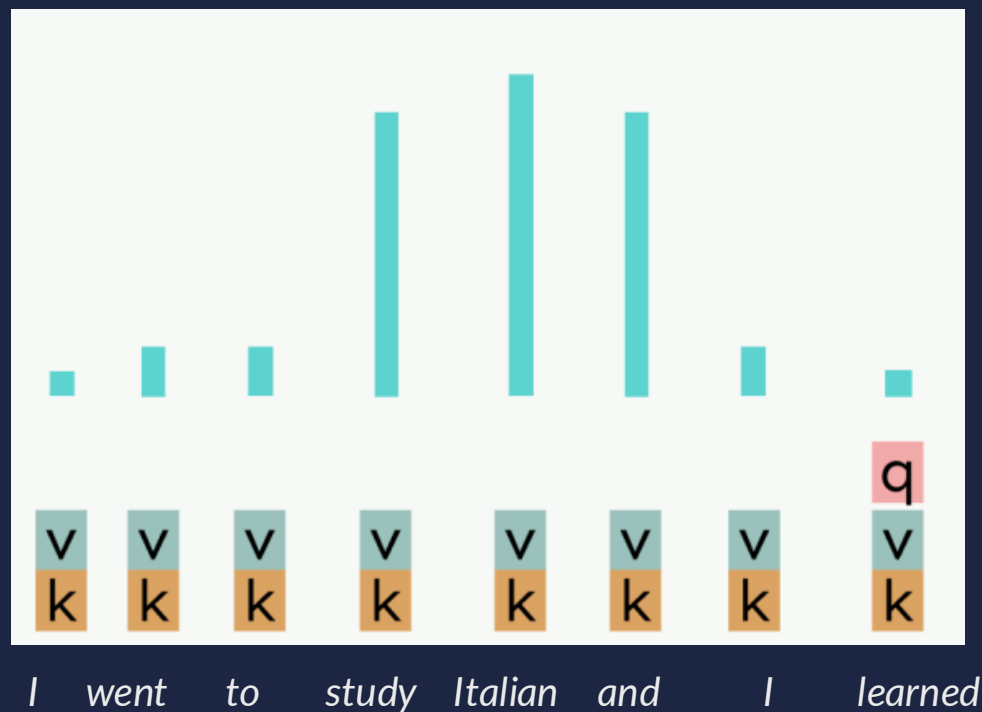
Recall our earlier example: *"I went to study Italian and I learned"*. In standard self-attention, the query for "learned" yields a single probability distribution to gather context. But language is complex. The word "learned" needs to interact with the *\*action\** ("study"), the *\*subject\** ("I"), and the *\*topic\** ("Italian") simultaneously.

A single attention mechanism struggles to route all these diverse semantic concepts. We need the ability to "look" in multiple directions at once.

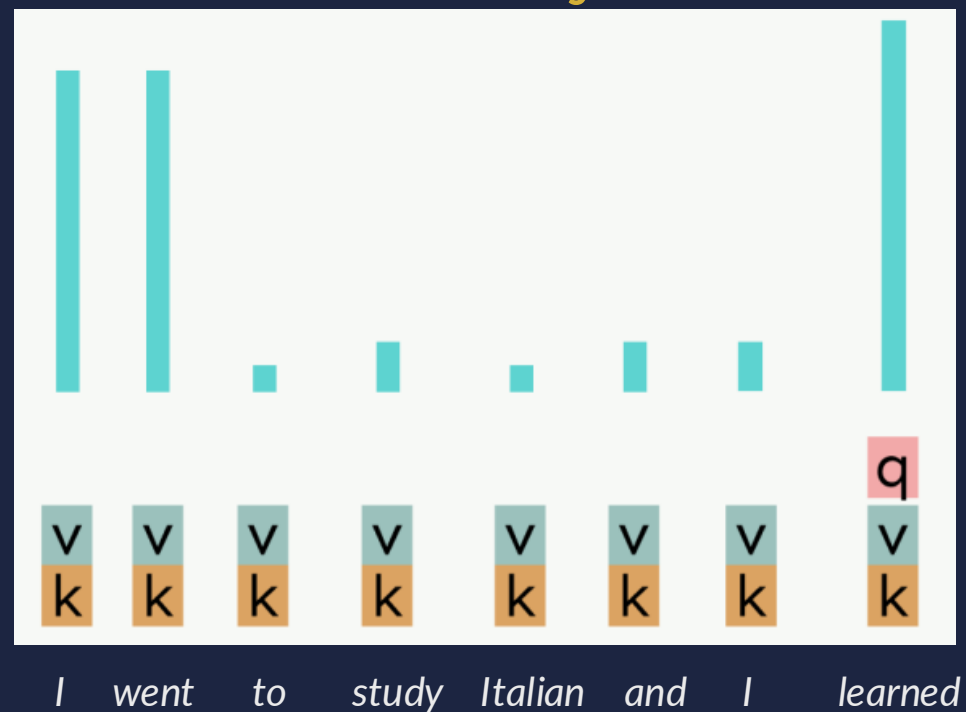


# Multi-Head Attention: Conceptual View

## Attention Head 1: Entities



## Attention Head 2: Syntax



# Sequence-Stacked Attention Matrices

First, take the query-key dot products in one matrix multiplication:  $XQ(XK)^T$

$$XQ \cdot K^T X^T = XQK^T X^T \in \mathbb{R}^{n \times n}$$

Next, softmax, and compute the weighted average with another matrix multiplication.

$$\text{softmax}\left(XQK^T X^T\right) \cdot XV = \text{output} \in \mathbb{R}^{n \times d}$$

## Vectorizing the Operations

To process sequences rapidly, we stack our individual token operations into massive matrices:  $XQ$ ,  $XK$  and  $XV$ .

The complete output sequence is computed simultaneously via a highly optimized matrix multiplication.

$$\text{output} = \text{softmax}(XQ(XK)^T)XV \in \mathbb{R}^{n \times d}$$

# Mathematical Definition of Multiple Heads

- What if we want to look in multiple places in the sentence at once?
  - For word  $i$ , self-attention “looks” where  $x_i^\top Q^\top K x_j$  is high, but maybe we want to focus on different  $j$  for different reasons?
- We’ll define **multiple attention “heads”** through multiple Q,K,V matrices
- Let,  $Q_\ell, K_\ell, V_\ell \in \mathbb{R}^{d \times \frac{d}{h}}$ , where  $h$  is the number of attention heads, and  $\ell$  ranges from 1 to  $h$ .
- Each attention head performs attention independently:
  - $\text{output}_\ell = \text{softmax}(X Q_\ell K_\ell^\top X^\top) * X V_\ell$ , where  $\text{output}_\ell \in \mathbb{R}^{d/h}$
- Then the outputs of all the heads are combined!
  - $\text{output} = [\text{output}_1; \dots; \text{output}_h] Y$ , where  $Y \in \mathbb{R}^{d \times d}$
- Each head gets to “look” at different things, and construct value vectors differently.

# Computational Efficiency

- Even though we compute  $h$  many attention heads, it's not really more costly.
  - We compute  $XQ \in \mathbb{R}^{n \times d}$ , and then reshape to  $\mathbb{R}^{n \times h \times d/h}$ . (Likewise for  $XK, XV$ .)
  - Then we transpose to  $\mathbb{R}^{h \times n \times d/h}$ ; now the head axis is like a batch axis.
  - Almost everything else is identical, and the **matrices are the same sizes**.

First, take the query-key dot products in one matrix multiplication:  $XQ(XK)^\top$

$$XQ \cdot K^\top X^\top = XQK^\top X^\top \in \mathbb{R}^{3 \times n \times n}$$

3 sets of all pairs of attention scores!

Next, softmax, and compute the weighted average with another matrix multiplication.

$$\text{softmax}\left(XQK^\top X^\top\right) \cdot XV = \text{mix}(P) = \text{output} \in \mathbb{R}^{n \times d}$$

# Scaled Dot Product Attention

## The High-Dimensional Trap

As the dimensionality  $d$  grows, the magnitude of the dot products between queries and keys naturally inflates.

Extremely large dot product values push the softmax function into regions with near-zero, vanishing gradients, causing training to completely stall.

## The Scaling Fix

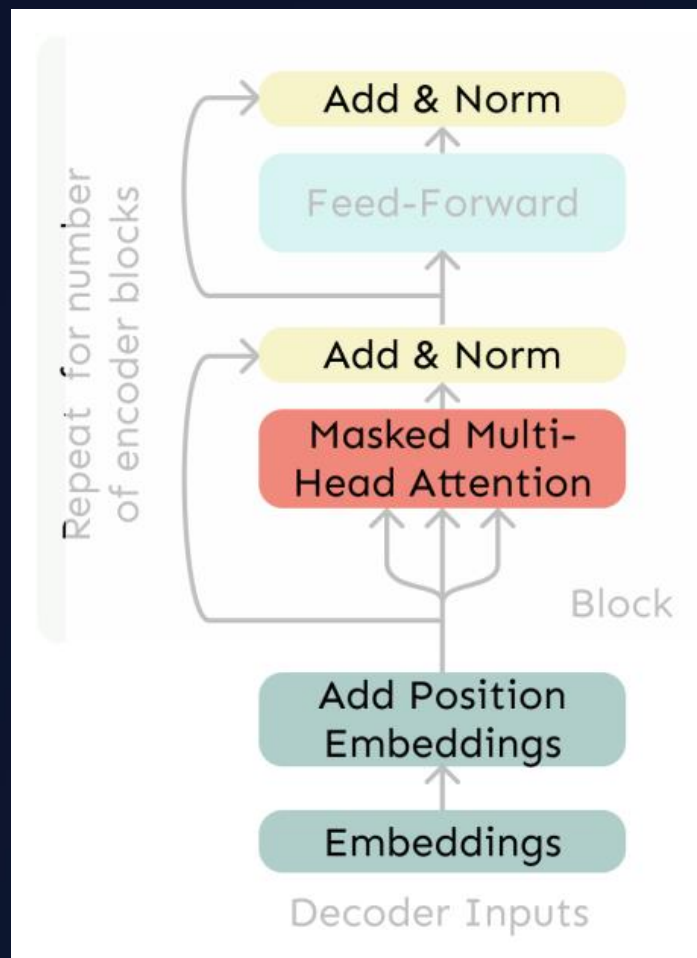
To stabilize the network, we scale the raw attention scores down by dividing them by the square root of the head dimension:  $\sqrt{d/h}$ .

$$\text{output}_\ell = \text{softmax}(XQ_\ell K_\ell^\top X^\top) * XV_\ell$$



$$\text{output}_\ell = \text{softmax}\left(\frac{XQ_\ell K_\ell^\top X^\top}{\sqrt{d/h}}\right) * XV_\ell$$

# Optimization Tricks: Add & Norm

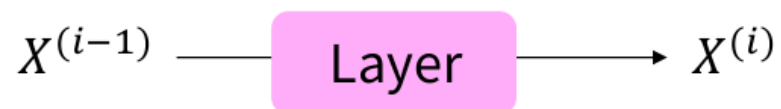


## Refining the Architecture

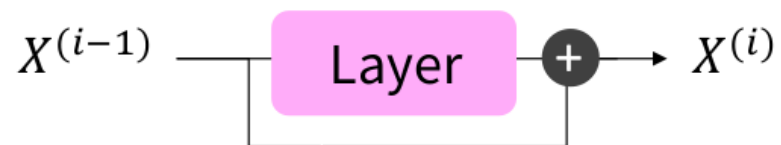
Returning to our Transformer Decoder diagram, you will notice crucial blocks labeled "**Add & Norm**" sandwiched between the major attention and feed-forward operations. These two mechanisms—Residual Connections and Layer Normalization—are not conceptually about sequence routing, but rather fundamental optimization tricks. Without them, deep networks of this size would suffer massive vanishing gradient collapse and fail to converge during training.

# The Power of Residual Connections

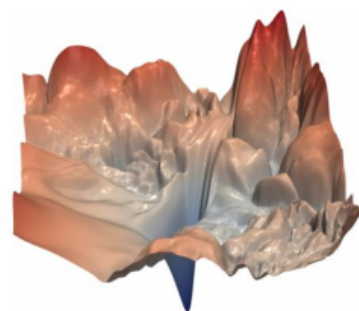
- **Residual connections** are a trick to help models train better.
  - Instead of  $X^{(i)} = \text{Layer}(X^{(i-1)})$  (where  $i$  represents the layer)



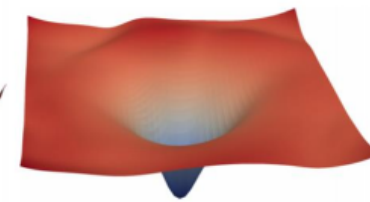
- We let  $X^{(i)} = X^{(i-1)} + \text{Layer}(X^{(i-1)})$  (so we only have to learn “the residual” from the previous layer)



- Gradient is **great** through the residual connection; it's 1!
- Bias towards the identity function!



[no residuals]



[residuals]

[Loss landscape visualization,  
[Li et al., 2018](#), on a ResNet]

# Optimization Trick: Layer Normalization

## Stabilizing Deep Networks

Layer Normalization is an essential trick designed to help these massive sequence models train faster and avoid catastrophic gradient failure.

It operates by taking the hidden vector values of a specific layer and mathematically normalizing them to possess a unit mean and standard deviation. This controls wild variance and uninformative spikes.

To retain representational capacity, the network learns two new parameters: Gain ( $\gamma$ ) to scale the normalized values, and Bias ( $\beta$ ) to shift them.

Let  $x \in \mathbb{R}^d$  be an individual (word) vector in the model.

Let  $\mu = \sum_{j=1}^d x_j$ ; this is the mean;  $\mu \in \mathbb{R}$ .

Let  $\sigma = \sqrt{\frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2}$ ; this is the standard deviation;  $\sigma \in \mathbb{R}$ .

Let  $\gamma \in \mathbb{R}^d$  and  $\beta \in \mathbb{R}^d$  be learned “gain” and “bias” parameters. (Can omit!)

Then layer normalization computes:

$$\text{output} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} * \gamma + \beta$$

Normalize by scalar mean and variance

Modulate by learned elementwise gain and bias

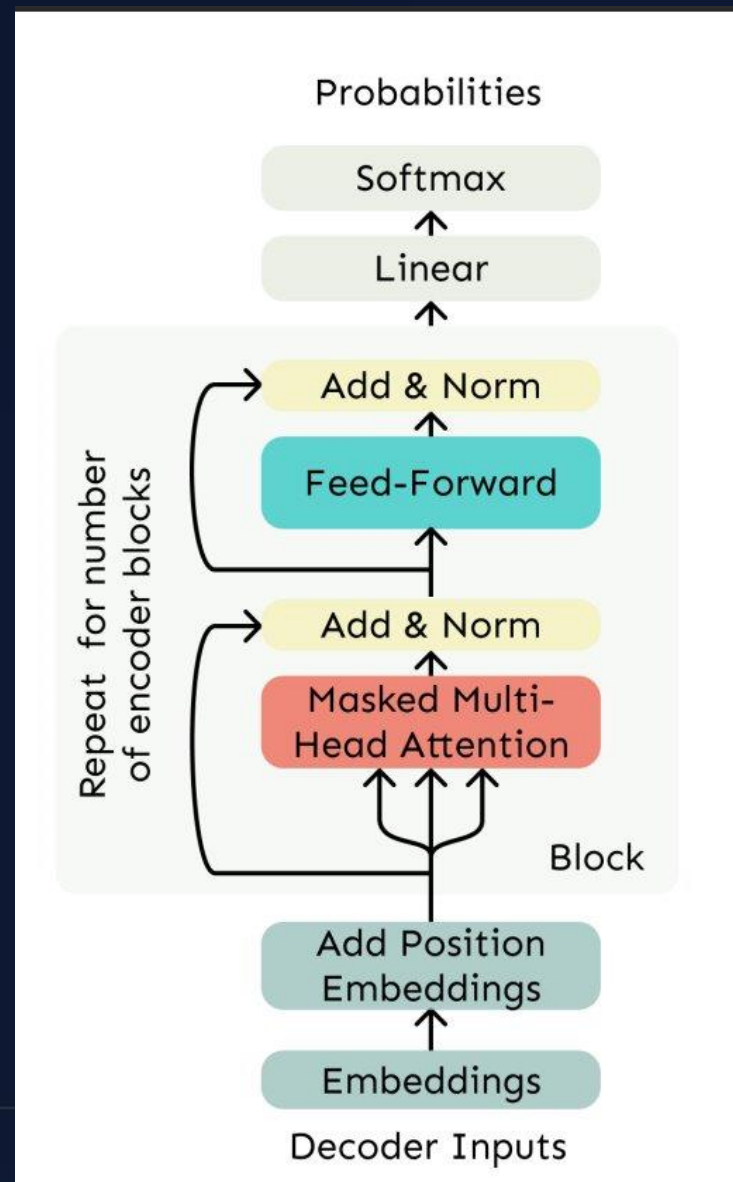
# The Complete Transformer Decoder Stack

The Transformer Decoder is a stack of Transformer Decoder Blocks.

• Each Block consists of:

- Self-attention
- Add & Norm
- Feed-Forward
- Add & Norm
- That's it!

We've gone through the Transformer Decoder.



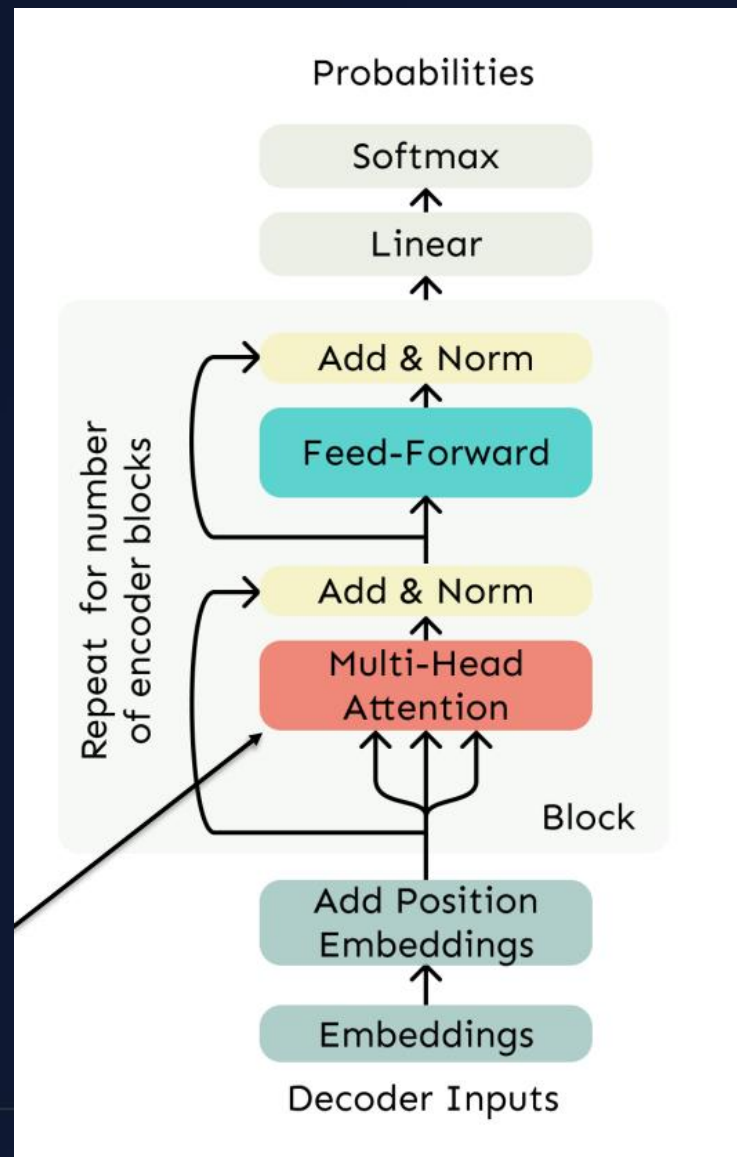
# Introducing the Transformer Encoder

The Transformer Decoder constrains to unidirectional context, as for language models.

What if we want bidirectional context, like in a bidirectional RNN?

This is the Transformer Encoder. The only difference is that we remove the masking in the self-attention.

NO MASKING!



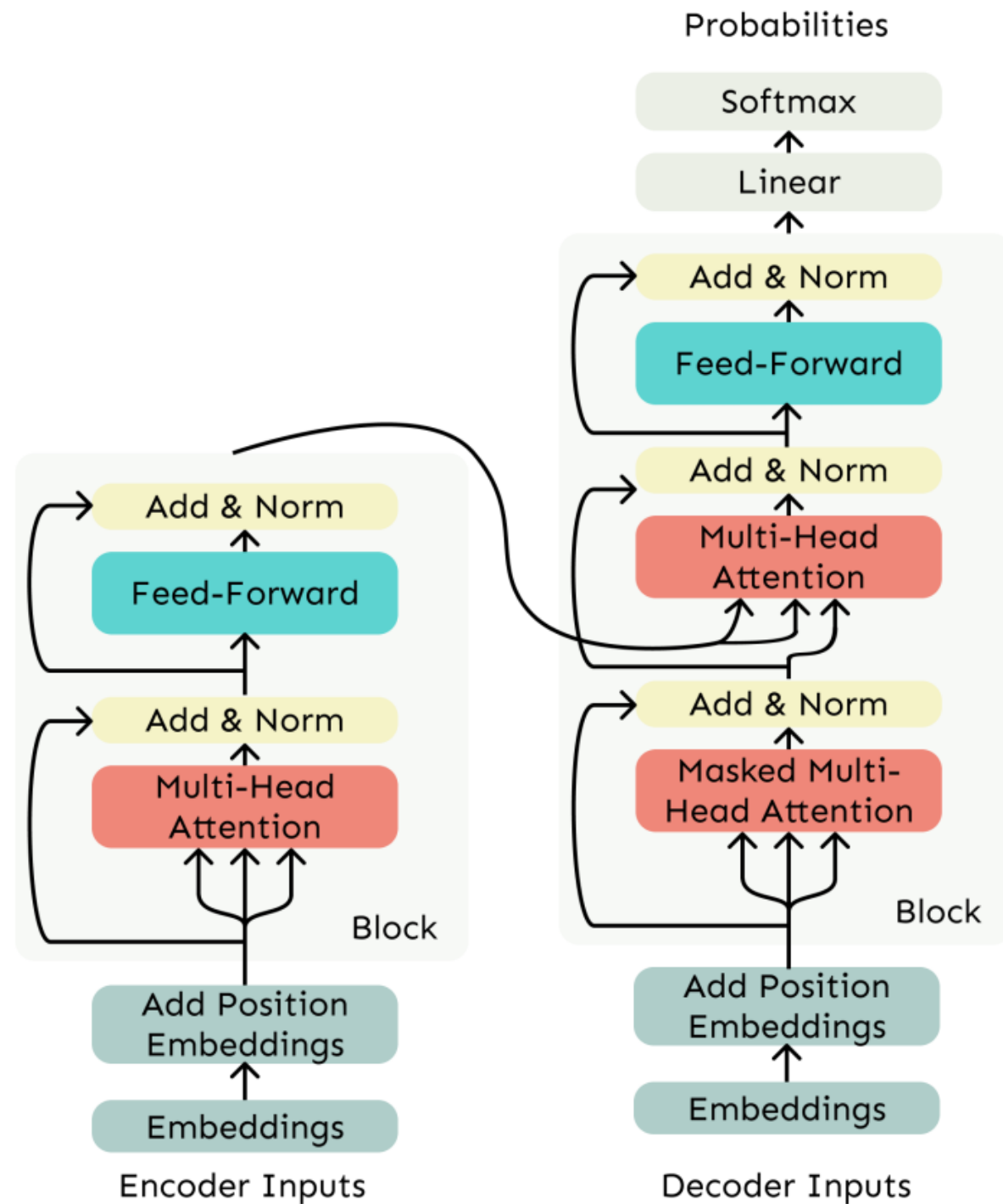
# The Full Encoder-Decoder Architecture

## Connecting the Modules

When solving sequence-to-sequence tasks like Machine Translation, we connect both modules together into a complete system.

The source text (e.g., French) is processed entirely bidirectionally by the Encoder stack, producing a massive, context-rich memory representation.

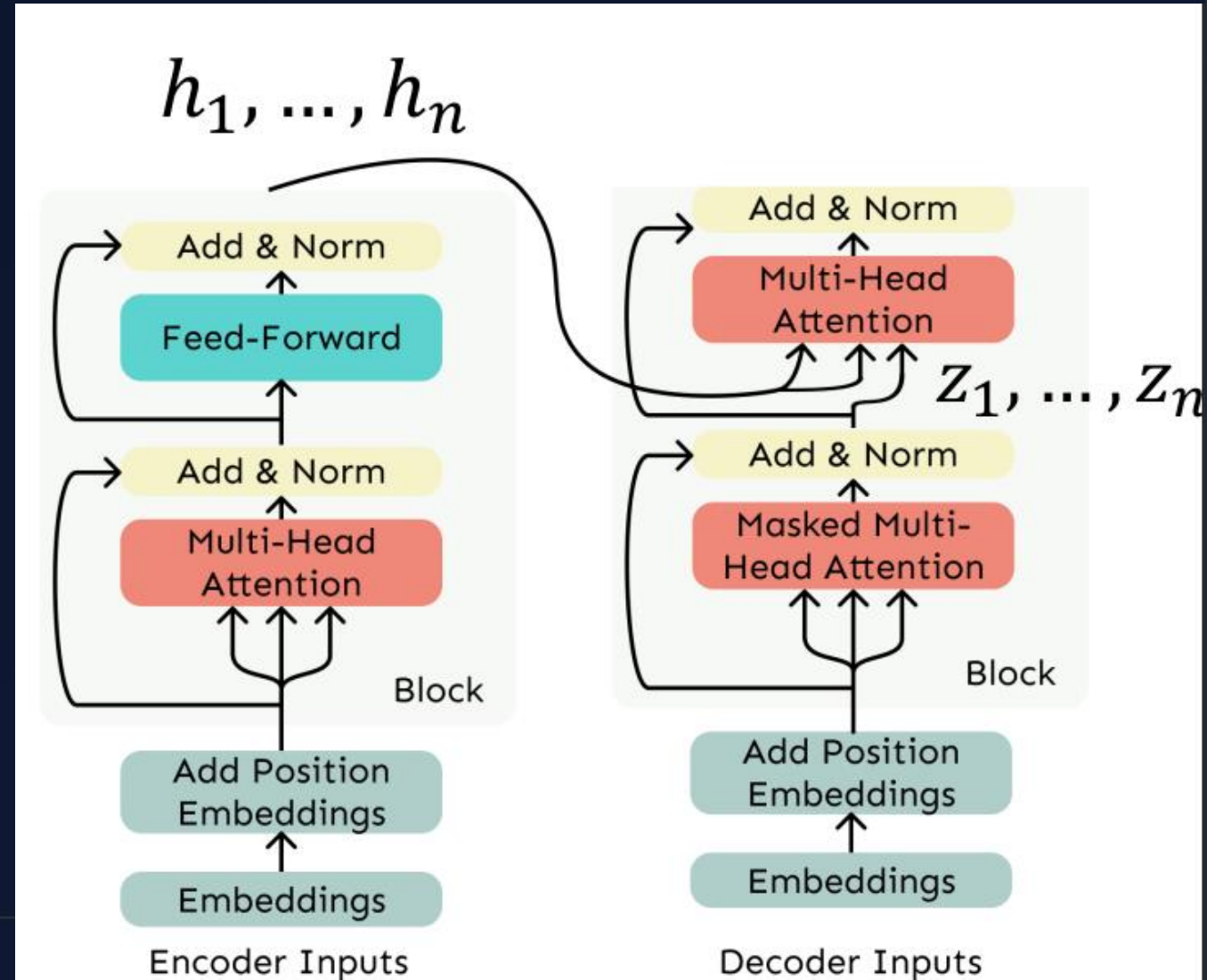
Simultaneously, the target text (e.g., English) is generated unidirectionally by the Decoder stack, which relies on a specialized mechanism called Cross-Attention to pull information from the Encoder's output.



# The Mechanics of Cross-Attention

## Bridging the Gap

Cross-attention operates exactly like self-attention, but its inputs are sourced from two distinct places. It acts as the bridge transferring data from the Encoder to the Decoder. In the Decoder's cross-attention layer, the **Queries** ( $q_i$ ) are drawn directly from the Decoder's previous layer, representing the current state of translation. However, the **Keys** ( $k_i$ ) and **Values** ( $v_i$ ) are drawn completely from the Encoder's final output sequence, acting as a static translation memory that the decoder continually references.



# Cross-Attention Mathematics

- Let's look at how cross-attention is computed, in matrices.
  - Let  $H = [h_1; \dots; h_T] \in \mathbb{R}^{T \times d}$  be the concatenation of encoder vectors.
  - Let  $Z = [z_1; \dots; z_T] \in \mathbb{R}^{T \times d}$  be the concatenation of decoder vectors.
  - The output is defined as  $\text{output} = \text{softmax}(ZQ(HK)^\top) \times HV$ .

First, take the query-key dot products in one matrix multiplication:  $ZQ(HK)^\top$

A diagram illustrating the first step of cross-attention. It shows a teal box labeled  $ZQ$  followed by an orange box labeled  $K^\top H^\top$ , with an equals sign and a grey box labeled  $ZQK^\top H^\top$ . To the right of the grey box is the text  $\in \mathbb{R}^{T \times T}$ . A blue text annotation "All pairs of attention scores!" points to the grey box.

$$ZQ \quad K^\top H^\top = ZQK^\top H^\top \in \mathbb{R}^{T \times T}$$

All pairs of attention scores!

Next, softmax, and compute the weighted average with another matrix multiplication.

A diagram illustrating the second step of cross-attention. It shows the word "softmax" followed by a large left square bracket, then a grey box labeled  $ZQK^\top H^\top$ , then a right square bracket, followed by a red box labeled  $HV$ , an equals sign, and a final grey box. To the right of the final grey box is the text "output  $\in \mathbb{R}^{T \times d}$ ". An arrow points from the grey box in the equation above to the  $ZQK^\top H^\top$  box in this equation.

$$\text{softmax} \left( ZQK^\top H^\top \right) HV = \text{output} \in \mathbb{R}^{T \times d}$$

---

# Great Results with Transformers

Examining the empirical dominance of the new  
paradigm.

---

# Translation Superiority

| Model                           | BLEU  |              | Training Cost (FLOPs) |                     |
|---------------------------------|-------|--------------|-----------------------|---------------------|
|                                 | EN-DE | EN-FR        | EN-DE                 | EN-FR               |
| ByteNet [18]                    | 23.75 |              |                       |                     |
| Deep-Att + PosUnk [39]          |       | 39.2         |                       | $1.0 \cdot 10^{20}$ |
| GNMT + RL [38]                  | 24.6  | 39.92        | $2.3 \cdot 10^{19}$   | $1.4 \cdot 10^{20}$ |
| ConvS2S [9]                     | 25.16 | 40.46        | $9.6 \cdot 10^{18}$   | $1.5 \cdot 10^{20}$ |
| MoE [32]                        | 26.03 | 40.56        | $2.0 \cdot 10^{19}$   | $1.2 \cdot 10^{20}$ |
| Deep-Att + PosUnk Ensemble [39] |       | 40.4         |                       | $8.0 \cdot 10^{20}$ |
| GNMT + RL Ensemble [38]         | 26.30 | 41.16        | $1.8 \cdot 10^{20}$   | $1.1 \cdot 10^{21}$ |
| ConvS2S Ensemble [9]            | 26.36 | <b>41.29</b> | $7.7 \cdot 10^{19}$   | $1.2 \cdot 10^{21}$ |

## The Vaswani 2017 Results

In the original paper "Attention Is All You Need", the proposed Transformer architecture completely decimated existing benchmarks.

It achieved substantially higher BLEU scores in major Machine Translation tasks, specifically English-to-German and English-to-French.

Crucially, it accomplished these state-of-the-art results at merely a fraction of the required training computational cost (FLOPs) compared to older ensemble models and recurrent networks.

# Advancing Document Generation

| Model                                                         | Test perplexity | ROUGE-L |
|---------------------------------------------------------------|-----------------|---------|
| <i>seq2seq-attention, <math>L = 500</math></i>                | 5.04952         | 12.7    |
| <i>Transformer-ED, <math>L = 500</math></i>                   | 2.46645         | 34.2    |
| <i>Transformer-D, <math>L = 4000</math></i>                   | 2.22216         | 33.6    |
| <i>Transformer-DMCA, no MoE-layer, <math>L = 11000</math></i> | 2.05159         | 36.2    |
| <i>Transformer-DMCA, MoE-128, <math>L = 11000</math></i>      | 1.92871         | 37.9    |
| <i>Transformer-DMCA, MoE-256, <math>L = 7500</math></i>       | 1.90325         | 38.8    |

The old standard



Transformers all the way down.



# Advancing Document Generation

## Lower Perplexity

Beyond translation, Transformer variants rapidly displaced the old sequence-to-sequence standard in document generation and language modeling tasks. They demonstrated massively improved Test Perplexity, indicating the models were far less "confused" by complex linguistic structures and long-range dependencies.

## Higher ROUGE-L

Simultaneously, these models achieved significantly higher ROUGE-L scores in text summarization. This proved that the generated texts were vastly superior at retaining the original contextual meaning and overlap with human reference summaries compared to recurrent predecessors.

# Total Benchmark Dominance

## The GLUE Leaderboard

The General Language Understanding Evaluation (GLUE) benchmark tests models across a wide array of difficult NLU tasks.

A quick glance at the leaderboard reveals absolute dominance by Transformer-based architectures. Top-ranking models like DeBERTa, MacALBERT, and StructBERT have pushed human-level baselines.

This success is driven by combining the highly parallelizable nature of the Transformer block with massive, internet-scale pretraining objectives.

| Rank | Name                     | Model                 | URL                 | Score                  |
|------|--------------------------|-----------------------|---------------------|------------------------|
| 1    | DeBERTa Team - Microsoft | DeBERTa / TuringNLRv4 | <a href="#">🔗</a>   | 90.8                   |
| 2    | HFL iFLYTEK              | MacALBERT + DKM       |                     | 90.7                   |
| +    | 3                        | Alibaba DAMO NLP      | StructBERT + TAPT   | <a href="#">🔗</a> 90.6 |
| +    | 4                        | PING-AN Omni-Sinitic  | ALBERT + DAAF + NAS | 90.6                   |
| 5    | ERNIE Team - Baidu       | ERNIE                 | <a href="#">🔗</a>   | 90.4                   |
| 6    | T5 Team - Google         | T5                    | <a href="#">🔗</a>   | 90.3                   |

---

# Drawbacks and Variants of Transformers

Understanding the limitations and engineering around the bottleneck.

---

# Primary Transformer Drawbacks



## Training Instabilities

Deep Transformer models are notoriously finicky to optimize. Improper normalization placement and scaling can lead to severe training instabilities and gradient collapse during the early phases of training.



## Quadratic Compute Cost

The core self-attention mechanism fundamentally requires the model to compute pairwise interactions between every single token in the sequence. This creates a massive mathematical bottleneck.

# Architectural Optimization: Pre-LN vs Post-LN

## The Post-LN Problem

The original BERT architecture utilized a "Post-LN" structure, meaning Layer Normalization was applied *after* the residual addition.

This approach inherently disrupts the clean, uninterrupted residual signal path running through the depth of the network, contributing heavily to the training instabilities observed in early iterations.

## The Pre-LN Solution

To resolve this, almost all modern Language Models have pivoted to a "Pre-LN" architecture.

By applying Layer Normalization directly to the inputs *before* the transformation occurs, the main residual highway remains completely clear, yielding vastly superior stability.

The one thing everyone agree on!

# The Quadratic Computation Issue

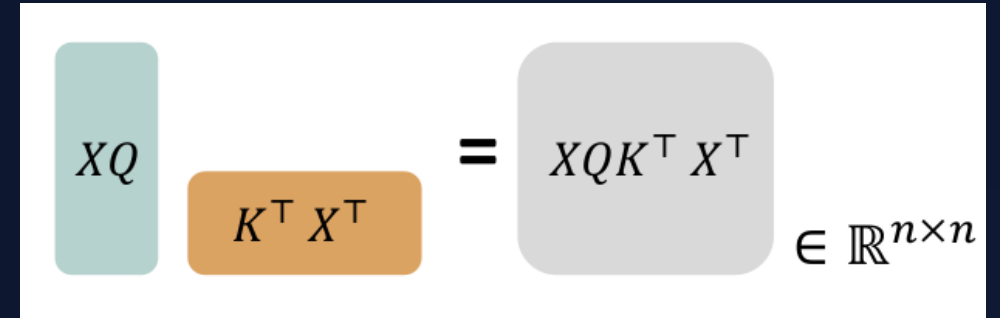
## The Mathematics of the Bottleneck

Because self-attention demands that every single word interact with every other word, the number of required calculations grows exponentially.

Specifically, the computational operations scale at:

$$O(n^2 d)$$

Where  $n$  is the sequence length. As sequences stretch past a few hundred tokens, this quadratic scaling becomes overwhelmingly expensive in terms of both memory and processing power.


$$XQ \cdot K^T X^T = XQK^T X^T \in \mathbb{R}^{n \times n}$$

Need to compute all  
pairs of interactions!  
 $O(n^2 d)$

# RNNs are Back

## Linear Scaling Alternatives

Due to the quadratic ceiling of Transformers, the industry has seen a massive resurgence of interest in Recurrent Neural Networks.

Modern State Space Models and RNN architectures, such as **RWKV** and **Mamba**, have successfully engineered around the historical limitations of recurrence.

Because these new models compress history into a rolling state, they provide highly coveted *linear scaling* for sequence generation, solving the long-context problem natively without quadratic memory blowouts.

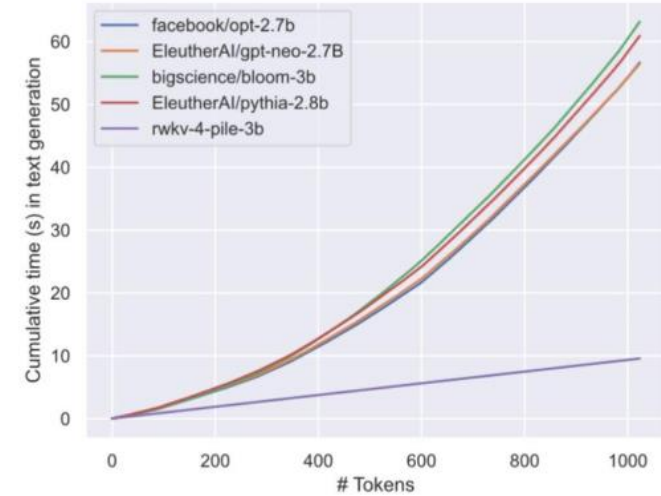
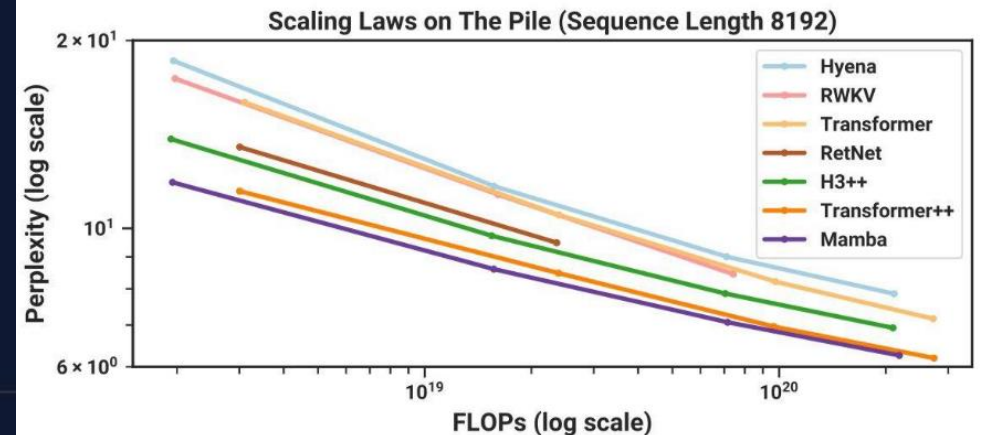


Figure 7: Cumulative time on text generation for LLM. Unlike transformers, RWKV exhibits linear scaling.



# Is Quadratic Cost a Dealbreaker?

||

Massive production models still rely on quadratic attention because system-level optimizations, like Flash Attention, have effectively mitigated the theoretical limits, pushing context windows to 100K+ tokens.

||

# Do Transformer Modifications Transfer?



Researchers have proposed hundreds of "Linear Attention" variants and complex architectural tweaks to circumvent the standard Transformer block.



Large-scale empirical analysis reveals a stark truth: the vast majority of these proposed modifications do not meaningfully transfer.



While they may perform well on highly specific or artificial tasks, they frequently fail to improve generalized downstream performance across broader NLU applications.



As a result, the standard, minimally modified Transformer (with optimizations like Pre-LN and ROPE) remains the undisputed anchor of the industry.

# Looking Ahead: Pretraining

Transitioning from architecture to the power of Foundation Models.

---